1. Somkamon Mettawiharee - st125881
2. Meta Puspa Maulida - st126320

EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('StudentsPerformance.csv')
```

```
df.head()
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1000 non-null   object
 1   race/ethnicity               1000 non-null   object
 2   parental level of education  1000 non-null   object
 3   lunch                        1000 non-null   object
 4   test preparation course      1000 non-null   object
 5   math score                   1000 non-null   int64
 6   reading score                1000 non-null   int64
 7   writing score                1000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.6+ KB
```

```python
df.describe()
```

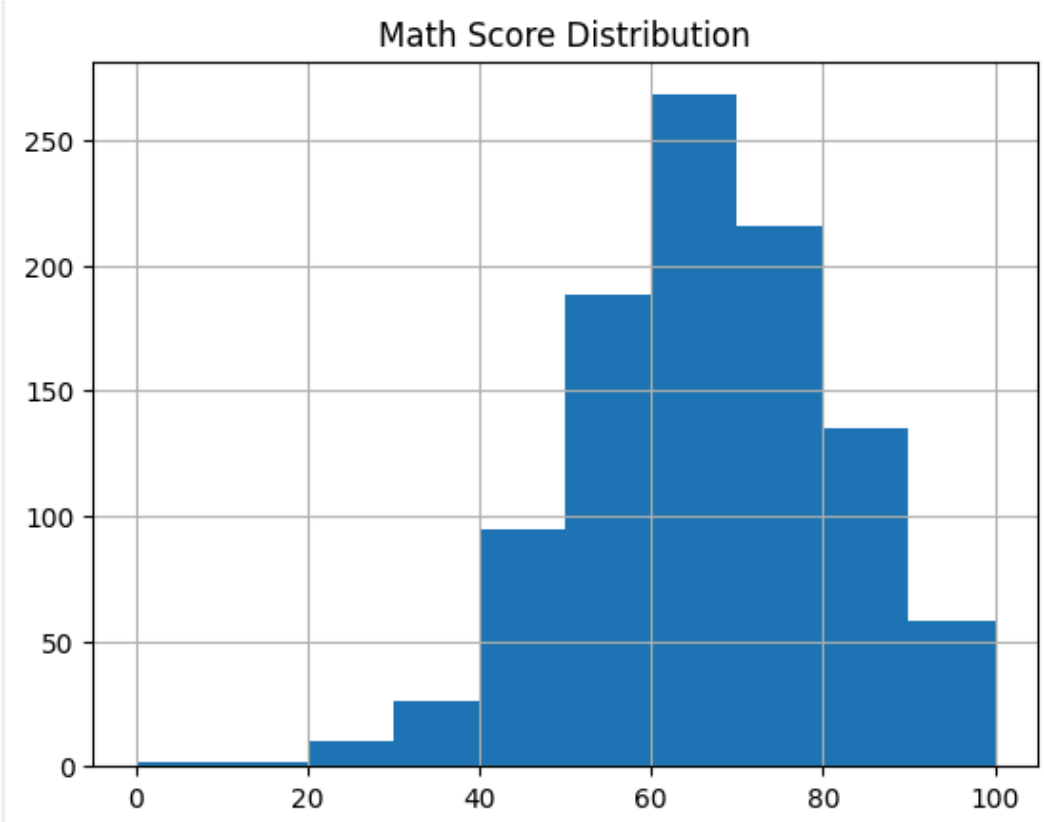
	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

```python
df.isnull().sum()
```

```
gender                       0
race/ethnicity               0
parental level of education  0
lunch                        0
test preparation course      0
math score                   0
reading score                0
writing score                0
dtype: int64
```

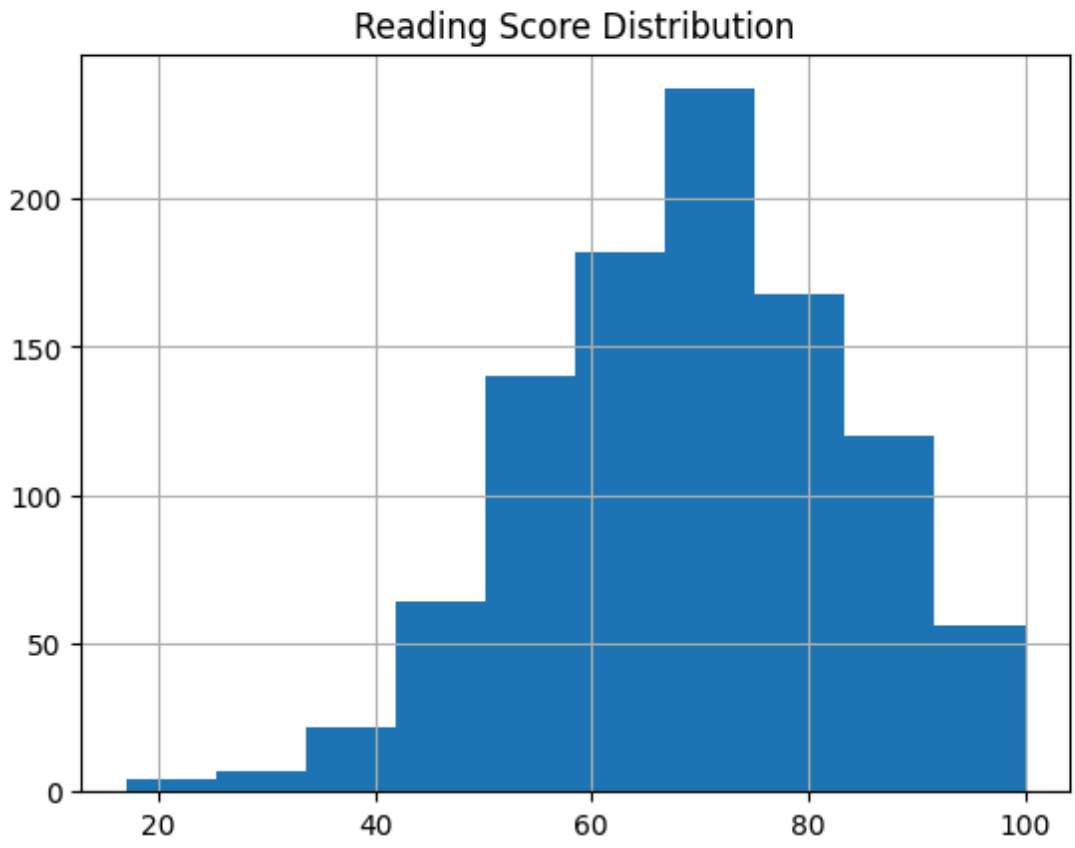
```python
df['average'] = (df['math score'] + df['reading score'] + df['writing score']) / 3

df['result'] = df['average'].apply(lambda x: 1 if x >= 60 else 0)
```

```
plt.figure()
df['math score'].hist()
plt.title("Math Score Distribution")
plt.show()
```



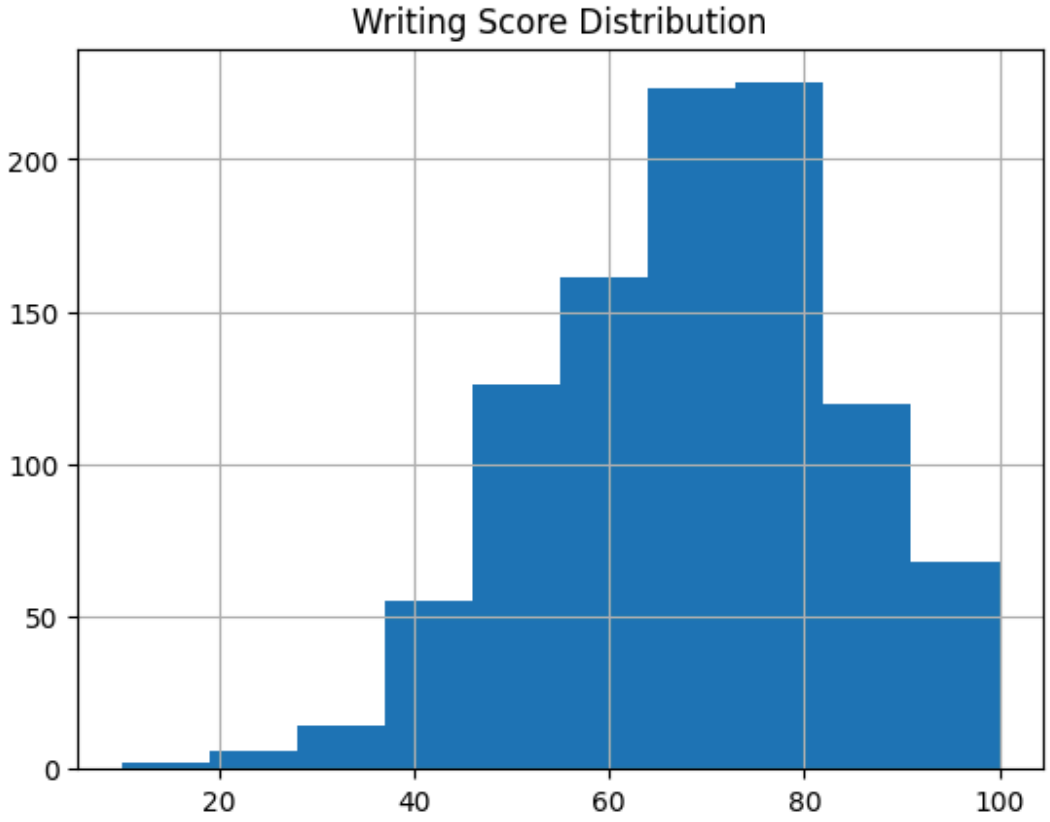
```
df['reading score'].hist()
plt.title("Reading Score Distribution")
```

```
Text(0.5, 1.0, 'Reading Score Distribution')
```

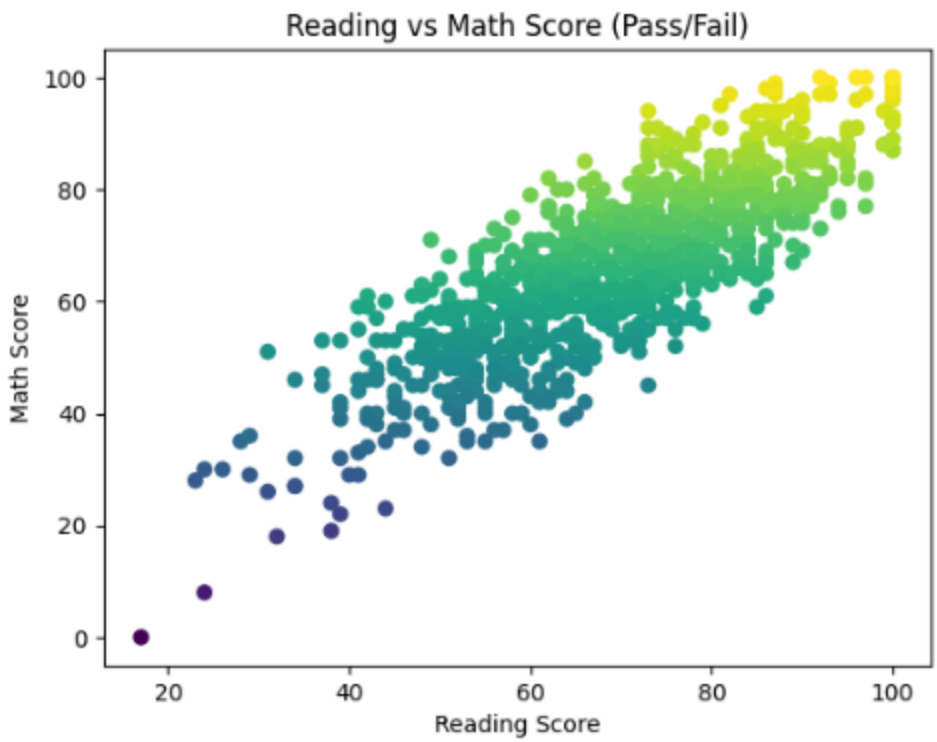


```
df['writing score'].hist()
plt.title("Writing Score Distribution")
```

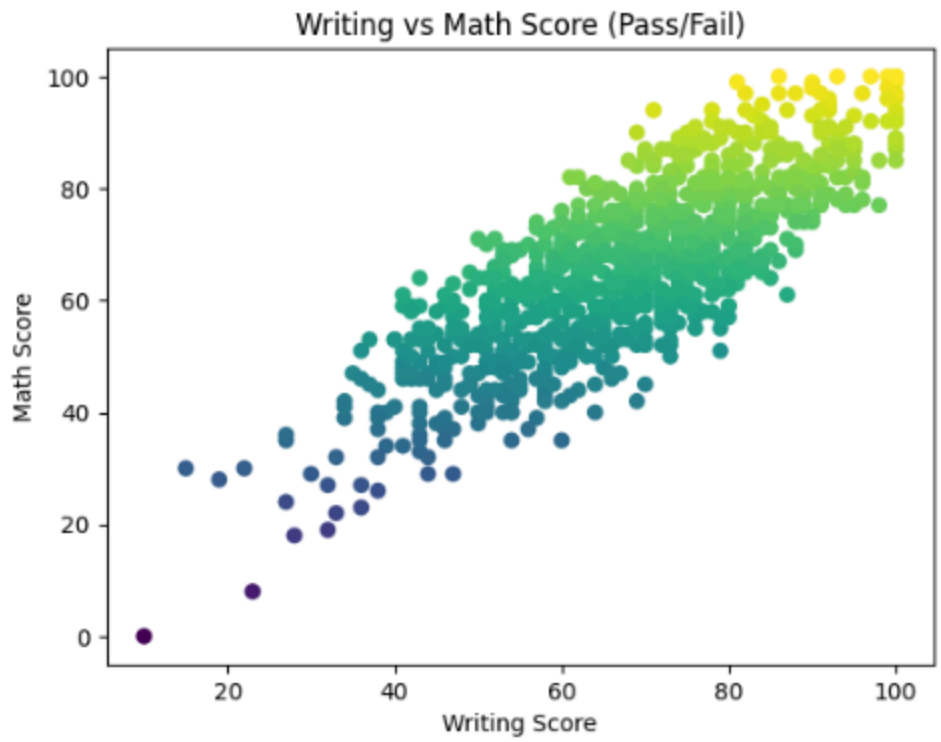
```
Text(0.5, 1.0, 'Writing Score Distribution')
```



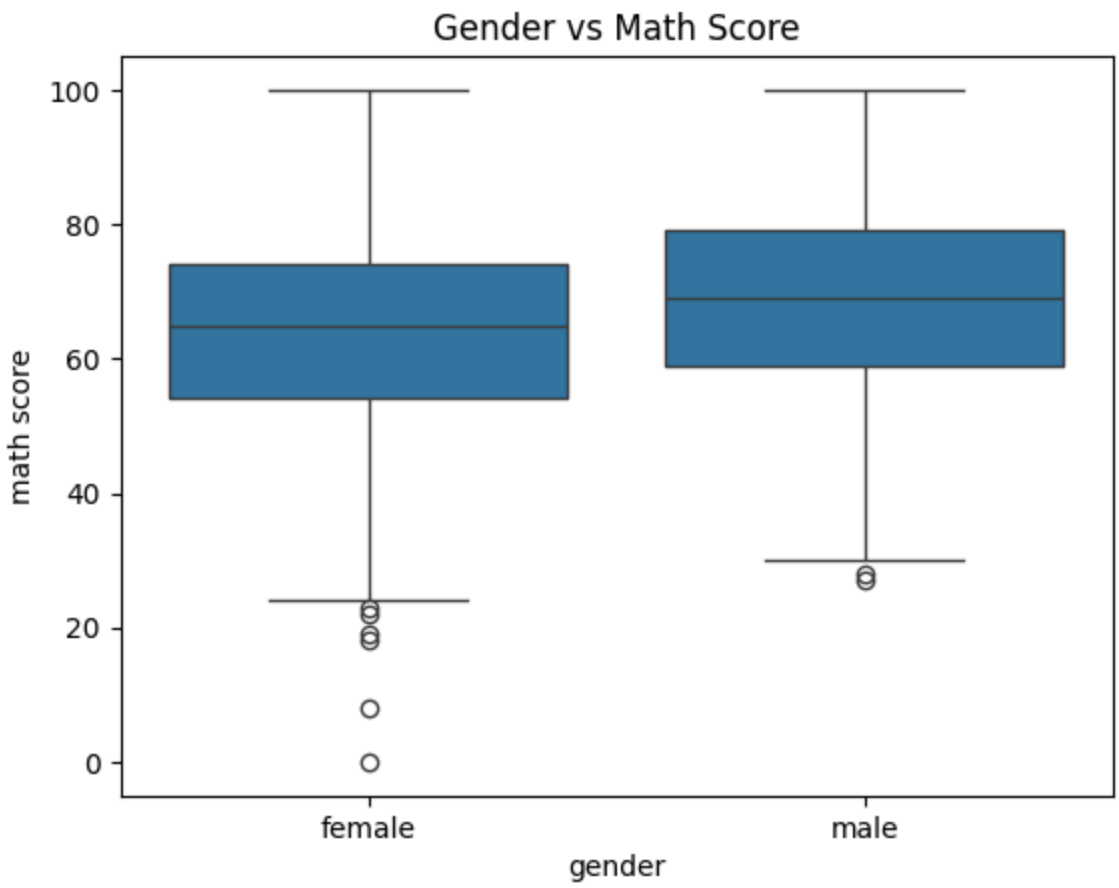
```python
import matplotlib.pyplot as plt

plt.figure()

plt.scatter(
    df['reading score'],
    df['math score'],
    c=df['math score']
)

plt.xlabel('Reading Score')
plt.ylabel('Math Score')
plt.title('Reading vs Math Score (Pass/Fail)')

plt.show()
```



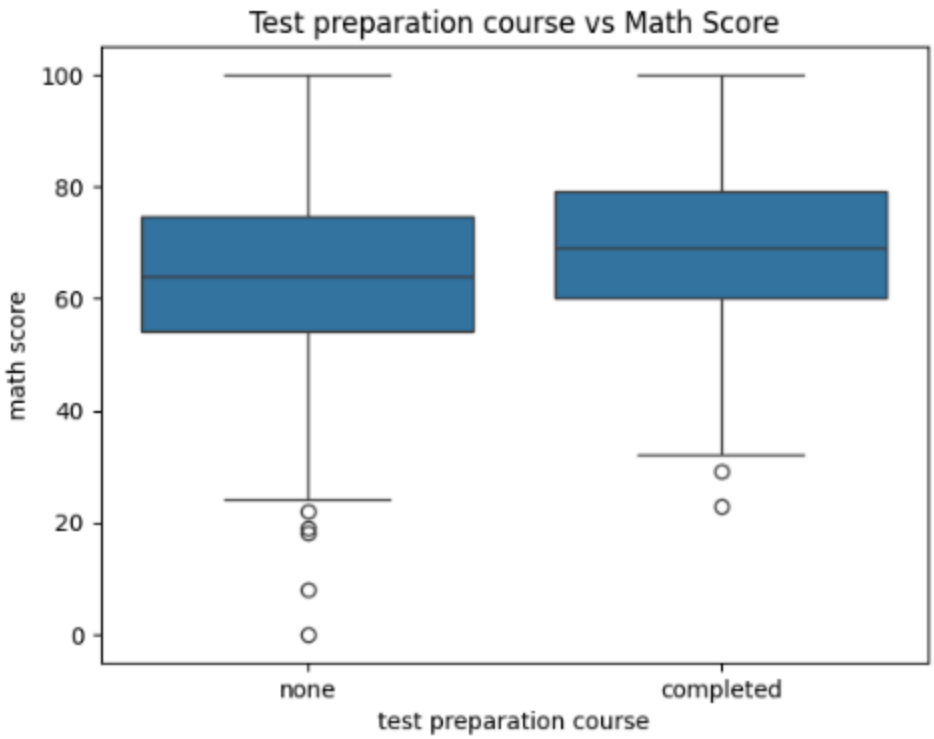
```
plt.figure()

plt.scatter(
    df['writing score'],
    df['math score'],
    c=df['math score']
)

plt.xlabel('Writing Score')
plt.ylabel('Math Score')
plt.title('Writing vs Math Score (Pass/Fail)')

plt.show()
```



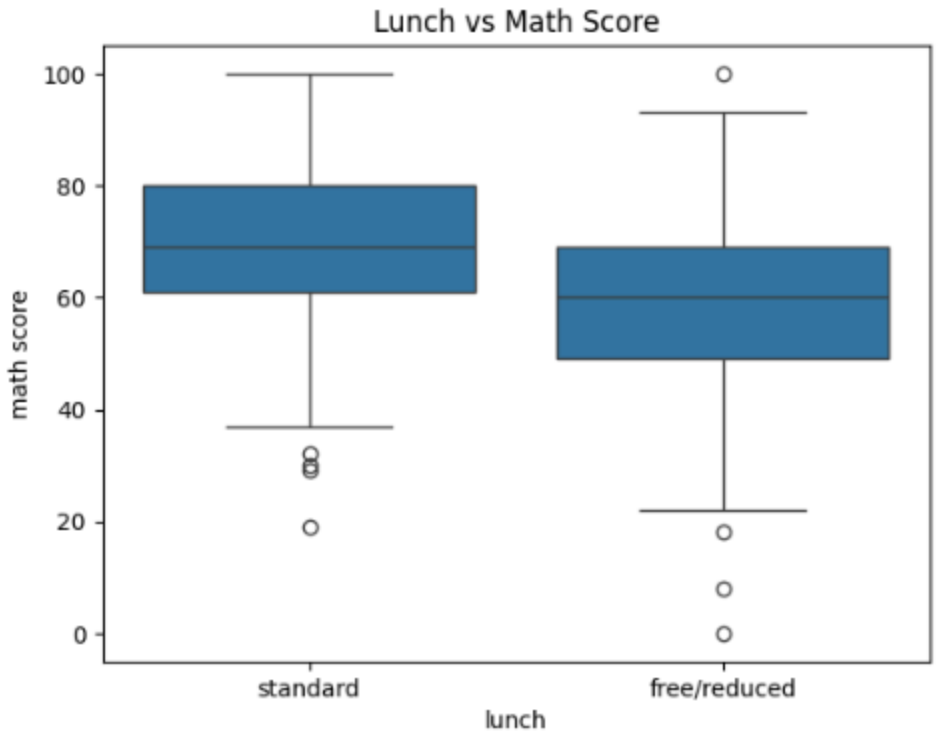
```
sns.boxplot(x='gender', y='math score', data=df)
plt.title("Gender vs Math Score")
plt.show()
```



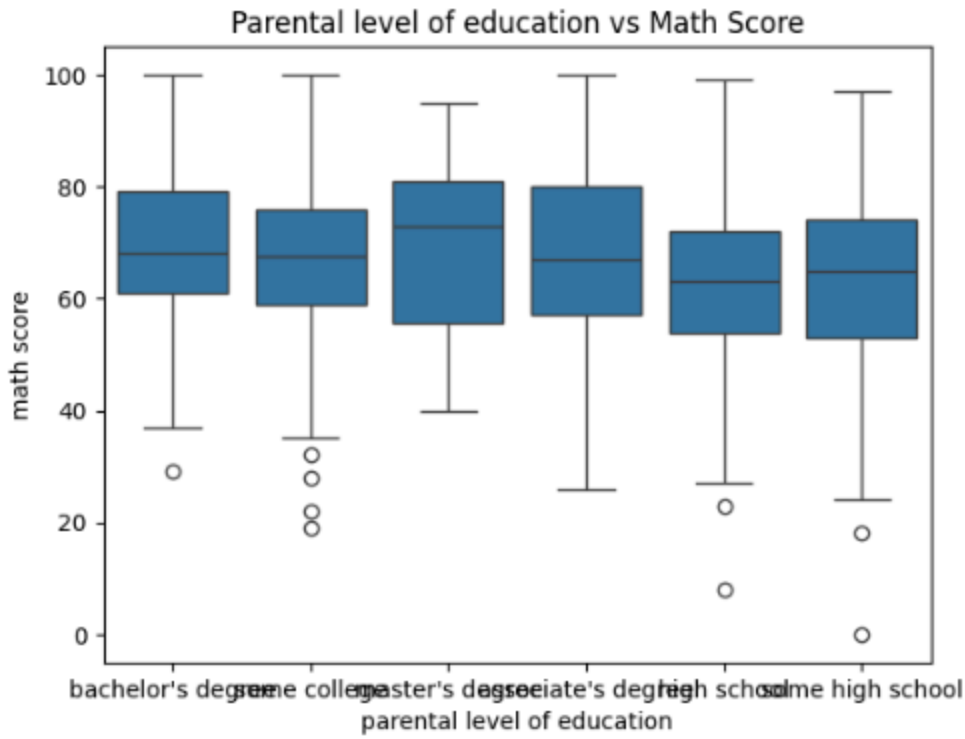
```
sns.boxplot(x='test preparation course', y='math score', data=df)
plt.title("Test preparation course vs Math Score")
plt.show()
```



```
sns.boxplot(x='lunch', y='math score', data=df)
plt.title("Lunch vs Math Score")
plt.show()
```



```
sns.boxplot(x='parental level of education', y='math score', data=df)
plt.title("Parental level of education vs Math Score")
plt.show()
```



Logistic Classification

- We use reading and writing scores as input features to classify whether a student will pass or fail in math exam

```
df['math_level'] = df['math score'].apply(lambda x: 1 if x >= 60 else 0)
```

```
#Logistic Classification model
#Select features (X) and target (y)
X = df[['reading score', 'writing score']]
y = df['math_level']
```

```
# Scale
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)
```

```
▾ LogisticRegression
▸ Parameters
```

```
y_pred = model.predict(X_test_scaled)
```

```python
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.825
```

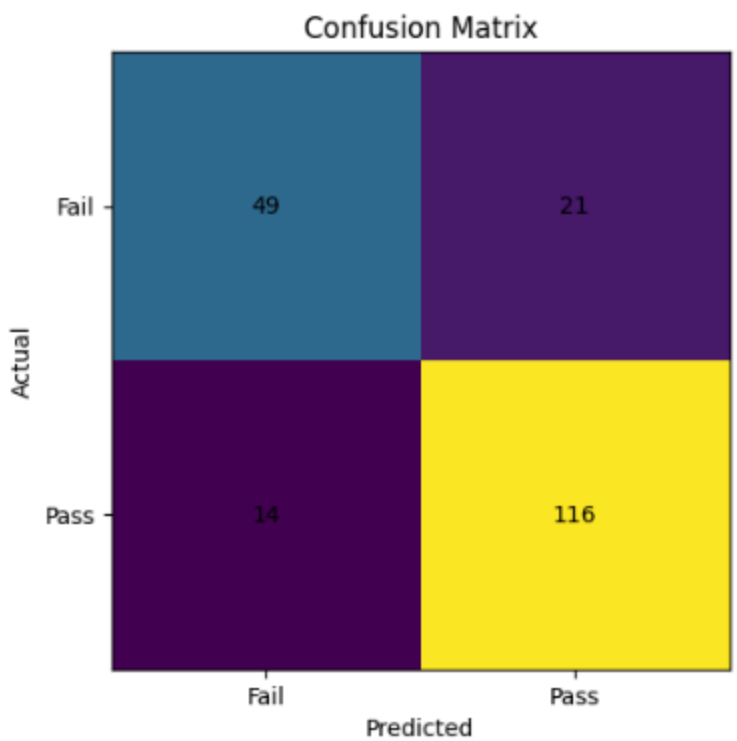
```python
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.78      0.70      0.74        70
           1       0.85      0.89      0.87       130

    accuracy                           0.82       200
   macro avg       0.81      0.80      0.80       200
weighted avg       0.82      0.82      0.82       200
```

```python
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[ 49  21]
 [ 14 116]]
```

```
plt.figure()
plt.imshow(cm)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.xticks([0,1], ['Fail','Pass'])
plt.yticks([0,1], ['Fail','Pass'])

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i,j], ha='center', va='center')

plt.show()
```



Testing data

```
#Test data
sample = [[100, 50]]  # reading, writing

sample_scaled = scaler.transform(sample)
prediction = model.predict(sample_scaled)

if prediction[0] == 1:
    print("Prediction: Pass")
else:
    print("Prediction: Fail")
```

```
Prediction: Pass
```

Docker progress

Group_Lab(2).ipynb

Using the logistic regression model from the previous

- Adding

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# Scale first
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Then train
model = LogisticRegression()
model.fit(X_scaled, y)
```

```
import pickle

pickle.dump(model, open("model.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))
```

Create [app.py]

```
from flask import Flask, request, render_template
import pickle
import numpy as np

app = Flask(__name__)

model = pickle.load(open("model.pkl", "rb"))
scaler = pickle.load(open("scaler.pkl", "rb"))

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        reading = float(request.form['reading'])
        writing = float(request.form['writing'])
```

```
        print("Reading:", reading)
        print("Writing:", writing)

        sample = [[reading, writing]]
        print("Sample:", sample)

        sample_scaled = scaler.transform(sample)
        print("Scaled:", sample_scaled)

        prediction = model.predict(sample_scaled)[0]
        print("Prediction:", prediction)

        result = "Pass" if prediction == 1 else "Fail"

        return render_template('index.html', prediction_text=f"Result:
{result}")

    except Exception as e:
        return f"ERROR: {str(e)}"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Pass/Fail Predictor</title>
</head>
<body>

<h2>Student Prediction</h2>

<form action="/predict" method="post">
    Reading Score: <input type="number" name="reading" required><br><br>
    Writing Score: <input type="number" name="writing" required><br><br>
    <button type="submit">Predict</button>
</form>

<h3>{{ prediction_text }}</h3>
```

```
</body>
</html>
```

Create Docker file

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY model.pkl .
COPY scaler.pkl .
COPY app.py .
COPY templates/ ./templates/

EXPOSE 5000

CMD ["python", "app.py"]
```

Build docker image and run

```
PS C:\Users\Somkamon\Documents\Machine learning\Group_Lab Deploy> docker build -t somkamon/hehehe:v1
.
```

☐	●	somkamon/hehehe	v1	15c3615c592a	18 seconds a(	657.36 MB

```
PS C:\Users\Somkamon\Documents\Machine learning\Group_Lab Deploy> docker run -p 5000:5000 somkamon/he
hehe:v1
```

```
WARNING: This is a development server. Do
I server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
```

After deploy to Server (SSH): http://192.41.170.112:5881/

Screen shot or result

Student Prediction

Reading Score: 100

Writing Score: 60

Predict

Result: Pass